

# Week 1 Reflection

---

## 1. In Coding Quiz 1, you are asked to find the distance of the farthest match in a set. Is this farthest match distance too far to be a meaningful match? How can you decide this?

Yes, a distance of the farthest match in a set is too great to be a meaningful match. The reason is that matching usually only works if the pair is similar. For example, pricing a kitchen remodel by comparing a remodeled house to one that has not been remodeled. This would be like comparing a house that is twice as large. The pricing difference would be about the size of the house, not the actual kitchen.

One way to decide is to compare the gap in the data spread. The furthest match is about 0.21 apart, which is about 0.73 standard deviations of Z. A common rule of thumb we can use allows for about 0.2 standard deviations. The typical match here is only 0.013 apart. This is due to 18 of the 48 treated rows having a larger Z than every control, so they all get matched to the same edge control.

```
CSVs = Path(__file__).parent / "csvs"
PERCENTILES = [50, 90, 95, 100]      # median, 90th, 95th, farthest

def load():
    """Split homework_1.2.csv into treated (X = 1) and control (X = 0) rows."""
    df = pd.read_csv(CSVs / "homework_1.2.csv")
    return df, df[df["X"] == 1], df[df["X"] == 0]

def nearest_matches(treated, control):
    """Approach A: nearest control on Z, with replacement. Returns distances and matched Y."""
    nn = NearestNeighbors(n_neighbors=1).fit(control[["Z"]])
    distances, indices = nn.kneighbors(treated[["Z"]])
    return distances.ravel(), control["Y"].to_numpy()[indices.ravel()]

def report_distances():
    df, treated, control = load()
    distances, _ = nearest_matches(treated, control)

    print(f"farthest match distance:      {distances.max():.4f}")
    print(f"median match distance:          {np.median(distances):.4f}")
    print(f"farthest, in SDs of Z:             {distances.max() / df['Z'].std():.2f} (rule of thumb: 0.2)")
    print(f"treated above every control: {(treated['Z'] > control['Z'].max()).sum()} of {len(treated)}")

    # The farthest match against the rest: is it typical or an outlier?
    print(f"\n{'percentile':<11} {'distance':>9}")
    for p, value in zip(PERCENTILES, np.percentile(distances, PERCENTILES)):
        print(f"{p:<11} {value:9.4f}")
```

The way we can be certain about this is to check whether the far matches actually change the answer. If we use all the matches, the effect is 0.543. If we keep only the matches within 0.05, it gives us 0.503. If we keep all the matches within 0.02, it gives us 0.500. The estimate improves once the bad matches are dropped, which proves that they were indeed the problem.

```

CALIPERS = [np.inf, 0.05, 0.02]      # inf keeps every match

def report_calipers():
    _, treated, control = load()
    distances, matched_y = nearest_matches(treated, control)
    treated_y = treated["Y"].to_numpy()

    # Drop matches farther than a caliper and see whether the effect moves.
    print(f"\n{'caliper':<9} {'matches kept':>13} {'effect':>8}")
    for caliper in CALIPERS:
        keep = distances <= caliper
        effect = (treated_y[keep] - matched_y[keep]).mean()
        print(f"{'caliper':<9} {'keep.sum():>13} {'effect:8.3f}")

```

## 2. Invent your own type of matching similar to (A) and (B), which has a different way to pick the matches in $X = 0$ .

The approach I would use that is similar to A and B would be weighted matching. Similar to approach B, I would take every control within a set of each treated row. The difference is that closer controls count more than the farther ones. A control right next to the treated row would be weighted more, whereas a control near the edge would not be weighted as much.

When we use approach A, it picks the control that is the closest, regardless of other controls being almost as close as the one that gets picked. When using approach B, it will look at every nearby control. Let's say there is one at 0.0 and another at 0.9. It would weight these equally, which doesn't really make sense.

Weighted matching is a balanced way of doing it. It would be like pricing a house where you look at houses that are nearby but put more weight on the ones that are very similar to each other.

```

RADIUS = 0.2

def radius_matching(treated, control, radius=RADIUS, weighted=True):
    """Each treated row vs. its controls within `radius`. weighted=False is approach B."""
    control_y = control["Y"].to_numpy()
    nn = NearestNeighbors(radius=radius).fit(control[["Z"]])
    dists, groups = nn.radius_neighbors(treated[["Z"]])
    effects = []
    for y, d, group in zip(treated["Y"].to_numpy(), dists, groups):
        if len(group):
            weights = 1 - d / radius if weighted else None
            effects.append(y - np.average(control_y[group], weights=weights))
    return np.mean(effects)

def report_matching():
    _, treated, control = load()
    _, matched_y = nearest_matches(treated, control)

    print(f"\n(A) nearest neighbor:      {(treated['Y'].to_numpy() - matched_y).mean():.3f}")
    print(f"(B) radius, equal weights:   {radius_matching(treated, control, weighted=False):.3f}")
    print(f"(C) radius, closer weighted: {radius_matching(treated, control, weighted=True):.3f}")

```

# Week 2 Reflection

---

## 1. Invent an example situation that would use fixed effects.

An example situation that would use fixed effects would be a coffee chain. Let's say the chain has 30 different locations. They roll out a new menu and want to know whether the new menu accounted for an increase in sales week over week.

The issue with this is that the stores were never comparable. For example, a location at an airport would sell a lot more than a location at a mall in a town in the middle of nowhere.

Really, what you would want to do is compare the stores across locations, not the actual menu. Using fixed effects would fix this issue by giving each store its own baseline. Each store would get its own intercept and the shared menu would be estimated on top of that. The question then becomes: did each store beat its own weekly sales?

## 2. Bootstrap the variance in the mean of a Pareto distribution. Explain what you had to do. As the sample size grows, what happens to that variance?

As the sample size grows, variance gets smaller. For example, think of Google reviews. A store with only a few Google reviews would be greatly impacted by one bad review or one good review. However, the more reviews a store gets, the less each individual review moves their average. This relates to Pareto values because the average becomes more stable.

To bootstrap the variance of the mean of a Pareto distribution, I took one sample from the distribution. After that, I made 5,000 new samples by randomly picking values, allowed repeating values, and then took the average of each. The variance of the mean is how much those 5,000 averages vary.

```
SHAPES = {"a = 3.0 (finite variance)": 3.0, "a = 1.5 (infinite variance)": 1.5}
SIZES = [100, 400, 1600, 6400, 25600] # quadruples, so 1/n predicts a ratio of 0.25
N_BOOTSTRAP = 5_000
N_REPLICATES = 40 # one sample alone is far too noisy on a heavy tail
SEED = 0

def pareto_sample(rng, n, a):
    """numpy's pareto() is Lomax and starts at 0, so shift it to start at 1."""
    return 1.0 + rng.pareto(a, n)

def bootstrap_mean_variance(sample, rng):
    """Resample with replacement B times, return the variance of the B means."""
    idx = rng.integers(0, len(sample), size=(N_BOOTSTRAP, len(sample)))
    return sample[idx].mean(axis=1).var(ddof=1)

def theoretical(a, n):
    """var(X)/n, which only exists for a > 2."""
    return a / ((a - 1) ** 2 * (a - 2)) / n if a > 2 else float("nan")

def run(label, a):
    rng = np.random.default_rng(SEED)
    print(f"\n{label}")
    print(f"{'n':>7} {'bootstrap var(mean)':>21} {'theory var(X)/n':>17} {'ratio vs prev':>14}")
```

```

previous = None
for n in SIZES:
    variance = float(np.mean([
        bootstrap_mean_variance(pareto_sample(rng, n, a), rng)
        for _ in range(N_REPLICATES)
    ]))
    ratio = "-" if previous is None else f"{variance / previous:.3f}"
    print(f"{n:>7} {variance:21.6f} {theoretical(a, n):17.6f} {ratio:>14}")
    previous = variance

if __name__ == "__main__":
    for label, shape in SHAPES.items():
        run(label, shape)
    print("\nSample size quadruples each row, so 1/n predicts a ratio near 0.25.")

```

Each time the sample size increased by four times, the variance decreased to about 25% of what it was before. This essentially means that the variance will shrink at about  $1/n$ . This in turn matches the Pareto theory. The one issue I found is that Pareto data sometimes will have big values, meaning a single run could seem scattered. Therefore I had to repeat each run 40 times and average those runs out.

# Week 3 Reflection

---

## 1. In the event study in Coding Quiz 3, how would we go about testing for a change in the second derivative as well?

The best way I thought of to go about testing for a change in the second derivative is to think of a car. The value is where the car is at a certain point in time. The first derivative would be its speed, and the second derivative would be whether it's speeding up or slowing down.

The quiz tested whether the car suddenly moved to a new spot in time and whether its speed changed. Testing the second derivative is simply asking whether the car sped up or slowed down at a certain point in time. In order to test it we can add an additional term to the regression equation. The term would be a squared term that only applies after the event. If that term is different from zero, that means the curve changed at that event.

I then ran this on the quiz data. In the Week 3 section of `Kyle_Reynolds_Weeks_1-4_Code.ipynb`, none of the three datasets actually showed a change in the second derivative, meaning the answer still holds.

## 2. Create your own scenario that illustrates differences-in-differences. Describe the story behind the data and show whether there is a nonzero treatment effect.

My own scenario that illustrates differences in differences would be a software company that adds an onboarding checklist. They add this checklist because they're hoping to reduce the number of support tickets. U.S. customers get this onboarding checklist right away, but EU customers don't get it until later.

Support tickets went down in the U.S. after the onboarding checklist launched, but they also went down in the EU, which never got the checklist. This means the drop in support tickets was not caused solely by the checklist.

In my simulated data, the number of U.S. support tickets dropped by 1.34 per month, and EU tickets dropped by 0.55 per month. The checklist did have a real effect:  $1.34 - 0.55 =$  about 0.78 fewer support tickets per month. This means that there is a non-zero treatment effect since the onboarding checklist actually reduced the number of support tickets.

```
N_ACCOUNTS = 200          # per region
MONTHS = 12
LAUNCH = 7                # first month with the checklist
TRUE_EFFECT = -0.8        # tickets per account per month
SEED = 0

def simulate(seed=SEED):
    """Tickets fall ~0.1/month in both regions as the product matures (parallel
    trends). US accounts start higher; the checklist cuts 0.8 more from launch."""
    rng = np.random.default_rng(seed)
    rows = []
    for us in (0, 1):
        baseline = 4.0 + 1.0 * us
        for account in range(N_ACCOUNTS):
```

```

        account_level = rng.normal(0, 1.0)          # some accounts just file more
        for month in range(1, MONTHS + 1):
            post = int(month >= LAUNCH)
            mean = baseline + account_level - 0.1 * month + TRUE_EFFECT * us * post
            rows.append({"account": f"{us}-{account}", "us": us, "month": month,
                        "post": post, "tickets": mean + rng.normal(0, 1.0)})
        return pd.DataFrame(rows)

def diff_in_diff(df):
    """tickets ~ us + post + us*post, SEs clustered by account (12 rows each)."""
    design = sm.add_constant(df[["us", "post"]].assign(us_post=df["us"] * df["post"]))
    return sm.OLS(df["tickets"], design).fit(
        cov_type="cluster", cov_kws={"groups": pd.factorize(df["account"])[0]})

def pre_trend(df):
    """Before launch only: does the US trend differ from the EU trend?"""
    pre = df[df["post"] == 0]
    design = sm.add_constant(pre[["us", "month"]].assign(us_month=pre["us"] * pre["month"]))
    return sm.OLS(pre["tickets"], design).fit(
        cov_type="cluster", cov_kws={"groups": pd.factorize(pre["account"])[0]})

def report_scenario():
    df = simulate()
    means = df.groupby(["us", "post"])["tickets"].mean().unstack()
    means.index = ["EU (control)", "US (treated)"]
    means.columns = ["before", "after"]
    means["change"] = means["after"] - means["before"]
    print("\n", means.round(3), sep="")

    naive = means.loc["US (treated)", "change"]
    manual = naive - means.loc["EU (control)", "change"]
    fit = diff_in_diff(df)
    lo, hi = fit.conf_int().loc["us_post"]
    trend = pre_trend(df)

    print(f"\nnaive US before/after:    {naive:.4f}")
    print(f"diff-in-diff (by hand):    {manual:.4f}")
    print(f"diff-in-diff (regression): {fit.params['us_post']:.4f} "
          f"se {fit.bse['us_post']:.4f}  t {fit.tvalues['us_post']:.2f} "
          f"95% CI [{lo:.3f}, {hi:.3f}]  p {fit.pvalues['us_post']:.1e}")
    print(f"pre-launch trend gap:      {trend.params['us_month']:.4f} "
          f"se {trend.bse['us_month']:.4f}  p {trend.pvalues['us_month']:.3f}")

```

# Week 4 Reflection

---

**1. The Coding Quiz gives two options for instrumental variables. For the second item (dividing the range of W into multiple ranges), explain how you did it, show your code, and discuss any issues you encountered.**

I broke the data into 20 groups based on W. That way, everyone in a group has about the same W. Then within each group, I found out how much Y changed when Z went from zero to one, and then divided it by how much X changed. After that, I took the average across 20 groups. I got 1.51, which is about the same as 1.56 from the first option. This means that controlling for W didn't change the answer. Therefore, we don't need W because Z was already random for everyone.

```
CSVS = Path(__file__).parent / "csvs"

def load(name):
    return pd.read_csv(CSVS / name)

def iv_effect(df):
    """(change in Y from Z=0 to Z=1) / (change in X from Z=0 to Z=1)."""
    means = df.groupby("Z")[["X", "Y"]].mean()
    change = means.loc[1] - means.loc[0]
    return change["Y"] / change["X"]

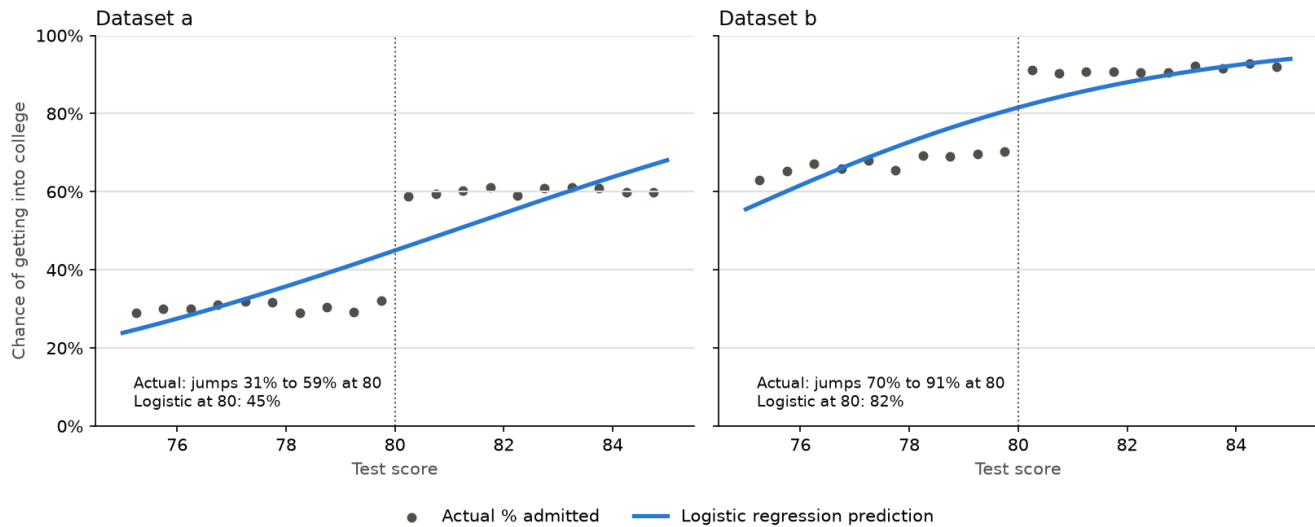
def effects_by_w(df, w_groups):
    """The effect within each W group. Groups with only one Z value are skipped."""
    return np.array([iv_effect(group) for _, group in df.groupby(w_groups, observed=True)
                     if group["Z"].nunique() == 2])

def report_instrument():
    df = load("homework_4.1.csv")
    print(f"effect ignoring W: {iv_effect(df):.2f}\n")

    # pd.cut: every group covers the same range of W. pd.qcut: every group has the same count.
    print(f"{'groups':<18} {'average':>8} {'lowest':>8} {'highest':>8}")
    for label, split, n in [("same range", pd.cut, 20), ("same range", pd.cut, 40),
                           ("same count", pd.qcut, 20), ("same count", pd.qcut, 100)]:
        effects = effects_by_w(df, split(df["W"], n))
        print(f"{'{label} x{n}':<18} {effects.mean():8.2f} {effects.min():8.2f} {effects.max():8.2f}")
```

The issue I ran into when I first split W into groups is that the groups at the edge of the ranges only had a few people in them. This is because the extreme W values are rare. I fixed this by using `pd.qcut`, which puts the same number of people into every group instead of breaking up the groups by range.

## 2. Plot the college outcome (Y) vs. the test score (X) in a small range of test scores around 80. On the plot, compare it with the Y probability predicted by logistic regression.



```

PLOT = Path(__file__).parent / "week4_reflection.png"
CUTOFF = 80
LOW, HIGH = CUTOFF - 5, CUTOFF + 5 # plot 75 to 85
DOT_WIDTH = 0.5 # each dot covers half a point of score

INK, MUTED, GRID, BLUE = "#0b0b0b", "#52514e", "#e5e4df", "#2a78d6"

def near_cutoff(name):
    df = load(name)
    df.columns = ["X", "Y"]
    return df[df["X"].between(LOW, HIGH)]

def share_admitted(df, low, high):
    return df.loc[(df["X"] >= low) & (df["X"] < high), "Y"].mean()

def draw(ax, df, title):
    # Dots: share admitted in each half-point of score.
    edges = np.arange(LOW, HIGH + DOT_WIDTH, DOT_WIDTH)
    dots = df.groupby(pd.cut(df["X"], edges, right=False), observed=True)["Y"].mean()
    ax.scatter(edges[:-1] + DOT_WIDTH / 2, dots * 100, s=22, color=MUTED, label="Actual % admitted")

    # Line: probability predicted by logistic regression.
    fit = smf.logit("Y ~ X", df).fit(dis=0)
    scores = pd.DataFrame({"X": np.linspace(LOW, HIGH, 200)})
    ax.plot(scores["X"], fit.predict(scores) * 100, color=BLUE, lw=2.5,
            label="Logistic regression prediction")

    # Label: actual share within one point on each side of 80 vs. the prediction at 80.
    below = share_admitted(df, CUTOFF - 1, CUTOFF)
    above = share_admitted(df, CUTOFF, CUTOFF + 1)
    at_cutoff = fit.predict(pd.DataFrame({"X": [CUTOFF]})).iloc[0]
    label = (f"Actual: jumps {below:.0%} to {above:.0%} at {CUTOFF}\n"
            f"Logistic at {CUTOFF}: {at_cutoff:.0%}")
    ax.text(LOW + 0.2, 5, label, color=INK, fontsize=9)
    print(f"{title}: {label.replace(chr(10), ' ')}")

```

```

ax.axvline(CUTOFF, color=MUTED, lw=1, ls=":")
ax.set_title(title, color=INK, loc="left")
ax.set_xlabel("Test score", color=MUTED)
ax.set_ylim(0, 100)
ax.yaxis.set_major_formatter(lambda v, _: f"{v:.0f}%")
ax.grid(axis="y", color=GRID, lw=1)
ax.spines[["top", "right"]].set_visible(False)

def report_plot():
    fig, axes = plt.subplots(1, 2, figsize=(11, 4.5), sharey=True)
    print()
    for ax, d in zip(axes, ["a", "b"]):
        draw(ax, near_cutoff(f"homework_4.2.{d}.csv"), f"Dataset {d}")

    axes[0].set_ylabel("Chance of getting into college", color=MUTED)
    fig.legend(*axes[0].get_legend_handles_labels(), loc="lower center", ncol=2, frameon=False)
    fig.tight_layout(rect=(0, 0.08, 1, 1))
    fig.savefig(PLOT, dpi=150)
    print(f"\nsaved {PLOT.name}")

```

The gray dots on the graphs are the actual data. Since Y is only ever 0 or 1, I grouped students with similar scores and then plotted the percent of each group that got into college.

The blue line is the logistic regression's prediction. Since it can only draw a smooth curve, it doesn't account for the jump at a test score of 80. In dataset A, the chance of getting in increases from ~30% to ~60% when you get a score of 80, but the logistic regression predicts 45%. Dataset B has the same exact problem. The real chance of getting in increases from ~70% to ~90%. But the logistic regression predicts about 80%.

This means that using plain logistic regression actually hides a jump, which is the most important part of this data.